

Given that AWS ALB **IP target mode** and **Pod Readiness Gates** are unsupported due to OVN-Kubernetes overlay networking 1, ROSA offers several robust architectural options for application ingress, load balancing, and rollout safety:

1. Built-in OpenShift Ingress Controller (HAProxy & Routes)

- **Mechanics:** ROSA includes a managed OpenShift Ingress Controller powered by HAProxy 2. Because the router runs inside the OVN-Kubernetes network, it communicates **directly with Pod overlay IPs** without NodePort translations 1, 3.
- **Capabilities:**
- Native OpenShift Route objects supporting **Edge, Passthrough, and Reencrypt** TLS termination 4.
- Built-in **AWS Shield** DDoS protection and a 10-second HAProxy timeout protection against slowloris-style attacks 5.
- Automated certificate management using the **cert-manager Operator** 6 or Let's Encrypt 7, and automated Route 53 hostname synchronization via the **External DNS Operator** 8.
- **Rollout Safety:** Zero-downtime deployments rely on standard Kubernetes readinessProbe definitions, PodDisruptionBudgets (PDBs) 9, and EndpointSlices. HAProxy dynamically monitors pod readiness and immediately removes unready pods from endpoints before terminating old replicas 10.

2. Fronted Edge Architecture (AWS ALB/NLB + OpenShift Ingress)

- **Mechanics:** If corporate policy requires AWS Certificate Manager (ACM) public certificates 11 or edge inspection via AWS WAF or CloudFront 12, place an internet-facing AWS load balancer in front of the private OpenShift Ingress Controller 13, 14.
- **Traffic Flow:** Internet → AWS ALB/NLB (ACM Cert / WAF) → Internal OpenShift Ingress (HAProxy) → Direct Pod IPs 13, 14.
- **Router Sharding:** You can deploy secondary IngressController Custom Resources with dedicated endpointPublishingStrategy types to isolate platform management listeners from public application routes 13, 14.

3. AWS Load Balancer Controller in Instance Mode (target-type: instance)

- **Mechanics:** You can deploy the AWS Load Balancer Controller to manage AWS Application Load Balancers via Kubernetes Ingress resources 13, 15.
- **Capabilities:**
- Must be configured explicitly with target-type: instance 1.
- Traffic routes from the ALB to EC2 worker node NodePorts, after which OVN-Kubernetes handles internal pod load balancing 1, 3.
- **Rollout Safety:** Managed via Kubernetes readinessProbe health checks, PodDisruptionBudgets (PDBs) 9, and rolling update deployment strategies rather than per-pod ALB readiness gates.

4. Layer 3 Load Balancing with MetalLB (BGP Mode) & Route Advertisements

- **Mechanics:** For baremetal worker pools or hybrid cloud topologies, ROSA supports the **MetalLB Operator** running in **BGP (Layer 3) mode** with **Bidirectional Forwarding Detection (BFD)** for sub-second failover 16-18.
- **Capabilities:**
- MetalLB speaker pods establish BGP peering sessions with upstream routers (such as AWS Transit Gateway or Direct Connect) 19, 20.
- OpenShift supports RouteAdvertisements and FRRConfiguration Custom Resources to dynamically advertise default cluster networks, Cluster User-Defined Networks (CUDNs), and EgressIPs over BGP 21-23.
- External routers distribute incoming client traffic across worker nodes statelessly using Equal-Cost Multi-Path (ECMP) routing 24.

5. Direct Ingress for OpenShift Virtualization VMs (Site-to-Site IPsec VPN)

- **Mechanics:** For non-Kubernetes VM workloads running on OpenShift Virtualization that require direct, non-NAT IP access from an AWS VPC, ROSA supports establishing a **Site-to-Site IPsec VPN** 25, 26.
- **Capabilities:**
- Deploys redundant gateway VMs running **Libreswan** and **Keepalived** inside a Cluster User-Defined Network (CUDN) 27, 28.
- Connects directly to an AWS **Transit Gateway (TGW)** using certificate-based authentication 27, allowing external VPC services to communicate directly with VM overlay addresses without requiring per-VM load balancers 26.

6. OpenShift Service Mesh (Istio)

- **Mechanics:** Deploys an Istio-based service mesh inside the cluster for east-west microservice traffic management 29, 30.
- **Capabilities:**
- Pod-level L7 traffic splitting (Canary deployments, Blue/Green rollouts) 29.
- Advanced resilience controls including circuit breaking, request retries, rate limiting, and mTLS between microservices 29, 30.

Ingress Option Summary

Architectural Pattern, Target Use Case, TLS Termination Point, Rollout Safety Mechanism
OpenShift Ingress (Routes), "Native OpenShift apps, custom domains 4", HAProxy Router (Edge / Reencrypt) 4, "readinessProbe + EndpointSlices + PDBs 9, 10"
Edge ALB + OpenShift Ingress, "ACM certificates 11, AWS WAF 12, Edge VPC", AWS ALB (Edge) 11 + HAProxy (Internal) 14, readinessProbe + HAProxy Endpoints 10
ALB Controller (instance), AWS-native ALB Ingress CRDs 13, AWS ALB 11, readinessProbe + RollingUpdate PDBs 9

MetalLB BGP Mode,"L3 baremetal ingress, TGW integration 19, 20",Upstream Router / Pod IP 19,ECMP 24 + BFD sub-second failover 17

Site-to-Site IPsec VPN,"Direct routable ingress to OpenShift VMs 25, 26",Libreswan IPsec Gateway VM 27,Keepalived Virtual IP Failover 28

OpenShift Service Mesh,Complex microservice traffic splitting 29,Envoy Ingress Gateway / Sidecars 30,L7 Traffic Shifting & Circuit Breaking 29

Would you like to examine an architectural manifest for configuring **Router Sharding** with secondary IngressController resources, or look into setting up the **External DNS Operator** with AWS Route 53?